Optimizing Texture Analysis for Improved AI-Generated Image Detection: An Enhanced PatchCraft Approach

Bain McHale
Language Technologies Institute
Carnegie Mellon University
Pittsburgh, PA 15213
dmchale@cs.cmu.edu

Juan Hernandez Gomez
Language Technologies Institute
Carnegie Mellon University
Pittsburgh, PA 15213
juanh@cs.cmu.edu

Sai Deepa Vaddi
Language Technologies Institute
Carnegie Mellon University
Pittsburgh, PA 15213
svaddi@cs.cmu.edu

Shamika Dhuri
Language Technologies Institute
Carnegie Mellon University
Pittsburgh, PA 15213
sdhuri@cs.cmu.edu

Abstract

In recent years, the rapid advancement of artificial intelligence models for image generation has made it challenging to distinguish real images from those created by AI. This difficulty has led to problems like the spread of fake news and the proliferation of unwanted deepfake content, emphasizing the need for effective detection methods. In this paper, we propose enhancements to the detection framework outlined in "PatchCraft: Exploring Texture Patch for Efficient AI-generated Image Detection." This framework identifies AI-generated images by analyzing inter-pixel correlations using certain preprocessing steps and a ResNet50 classifier over the input images. Our proposed enhancements include modifications to the classifier, employing various filter combinations, incorporating medium texture images, and eliminating the subtraction step. Experimental results suggest that omitting the subtraction step, reducing the number of original filters used, and integrating more advanced models like Inception can improve the accuracy of detecting AI-generated images with this method.

[1]

1 Introduction

Over the last few years, artificial intelligence models focused on image generation have experienced expeditious improvement. Since the creation of GANs [5] in 2014, many new models, such as Stable Diffusion [7], Midjourney, and DALL-E, have dramatically improved the quality of AI-generated images, making their detection challenging.

The number of deepfake images has risen sharply. This, along with the widespread concern caused by a fake image of an explosion in the Pentagon, underscores the need for effective and generalizable AI-generated image detection systems.

[1] Abstract written by Juan Hernandez Gomez (juanh)

Many authors have tackled this problem using different approaches, mainly focusing on binary classification. Nevertheless, most of these methods are not generalizable, considering they are trained as classification problems to detect images generated using specific models. Therefore, those solutions end up being inefficient for images generated with models outside of their scope. Moreover, with the current advances in AI, most models developed using this methodology have become obsolete quickly.

To address the generalization problem, most recent approaches have concentrated on finding a universal fingerprint to differentiate AI-generated images from real ones. Durall et al. [3] detected that up-sampling methods' inability to reproduce spectral distributions can be used to detect generated images. Bi et al. [2] found that generated images show abnormal behavior when projected on a dense subspace generated from real ones. Wang et al. [9] detected that diffusion-generated images can be approximately reconstructed by a pretrained diffusion model while real images cannot. Zhu et al. [14] used an adversarial teacher-student framework for anomaly detection in generated-images.

In the baseline model chosen for this project, Zhong et al. [12] present a more generalizable method than previous approaches. Their technique significantly enhances detection capabilities by leveraging the distinctive inter-pixel correlation differences between rich and poor texture areas in AI-generated images. The method involves preprocessing steps to strip semantic information from input images, followed by using a ResNet50 classifier to categorize the images as real or fake. This proposed approach surpasses several leading-edge detectors, achieving an approximate 4% improvement in average detection accuracy across 16 common generative models.

This report describes experiments performed on the framework initially proposed in the cited paper. The experiments used a dataset of labeled real and AI-generated images to measure accuracy and loss in both training and validation phases. The goal was to identify potential improvements to the original methodology. [2]

2 Literature Review

2.1 History of Literature

Unsurprisingly, the field of fake image recognition is deeply researched. Given that we have chosen the dataset which is a combination of Midjourney, DALL-E, and real images [1], which includes images of a large variety, we are going to focus on literature that is generalized to be for any type of image. Historically, there are many techniques that focus on detecting Computer Graphic (CG) Images [11], [6]. Such techniques predate fully AI-generated images, and almost exclusively rely on CNN classifiers. They have been able to leverage popular architectures like ResNet 50 with the integration of ideas like bagging the votes from patches of the images instead of analyzing the image as a whole.

2.2 Texture Analysis

As we move to more modern literature, though many still use CG identification techniques for identifying AI-generated images, there are models specifically designed for AI-generated images created using GANs and diffusion models. Many of these models focus on textures, as AI-generated images have difficulties replicating texture properly. A paper focusing on dual-stream networks utilizes residual streams to extract the texture from high-frequency areas of the image and content streams to extract the texture from low-to-mid frequency areas of the image before feeding them into a CNN [10] and it achieves an accuracy of 99.2% for a DALL-E2 based dataset. This approach can be further simplified to simply detect how texture-rich the image is. Zhong's paper on texture extraction shows that a simple approach of preprocessing the data using a high pass filter prior to feeding the images into a CNN classifier is also very effective [13]. We also see Zhong further use the idea of patching, indicating that it is effective not only for CG Images, but also for fully AI generated images and something to consider in any models going forward. The newest released version of the patch method is [12] which utilizes high-pass filters to extract the unique fingerprint to differentiate the AI generated images and the real images and achieves 89.55% for the Midjourney-based dataset and 96.60 % for the DALLE2-based dataset.

[2] Introduction written by Juan Hernandez Gomez (juanh)

2.3 Evaluating on Unseen Datasets

Another common issue raised in the literature is discriminators not generalizing well to new generative models. If a model is trained on GANs and then tested on images generated by diffusion models, it will perform poorly. As one might imagine, this is not ideal given that it means any new model becomes a zero-day attack against that discriminator. Adversarial student-teacher models can address this problem by exposing the teacher to the entire dataset but restricting certain generative model data from the student during training [14] and it achieves an accuracy of 89.6% for the Midjourney dataset. [3]

3 Model Description

3.1 Model Architecture

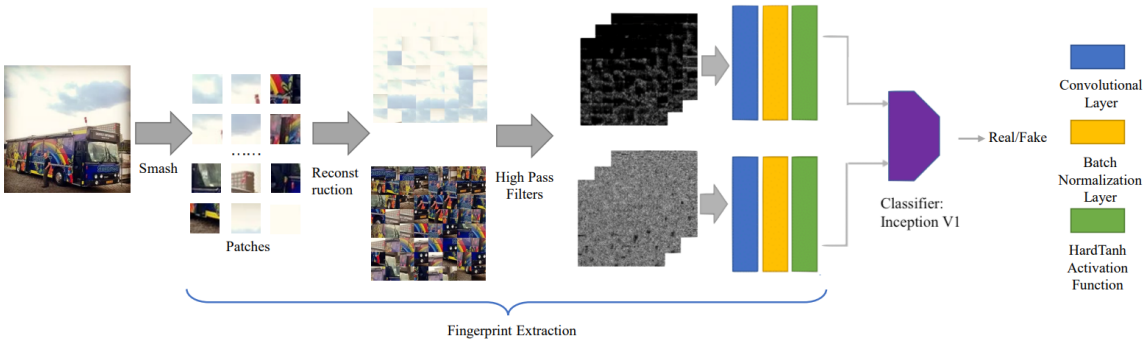


Figure 1: End-to-End workflow of the final model used.

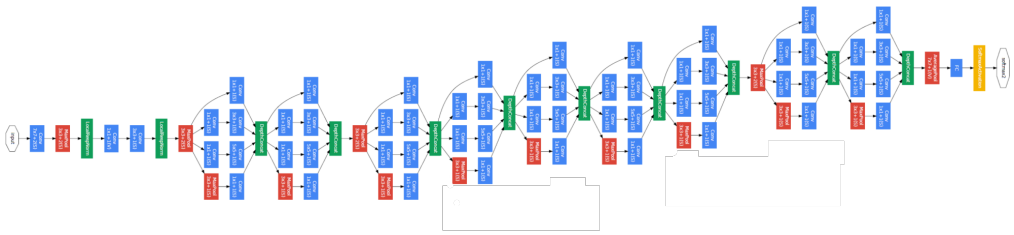


Figure 2: The implementation of Inception that was used as the classifier in our final code.

[3] Literature Review done by Shamika Dhuri (sdhuri), Juan Hernandez Gomez (juanh), Bain McHale (dmchale) and Sai Deepa Vaddi (svaddi). Section written by Sai Deepa Vaddi (svaddi).

Model	Hidden Layers	Parameters/Hyperparameters
Stem + ResNet-50	2x{CNN,Batch Norm, Hard Tanh}, 4x{CNN, Batch Norm, ReLU}, {CNN, Batch Norm, SiLU}, MaxPool, 5x{CNN, Batch Norm, SiLU}, CNN, Batch Norm, 4x{CNN, Batch Norm, SiLU}, CNN, Batch Norm, 4x{CNN, Batch Norm, SiLU}, CNN, Batch Norm, 3x{CNN, Batch Norm, SiLU}, Average Pool, Flatten, Linear	Initial Learning Rate: 2e-3 Batch size: 32 Epochs: 30 Size of Image Patches: 32 Number of Crops Made: 192 Number of Crops Used: 64 Subtract Textures: {True, False} Filters: [a, b, c, d, e, f, g, h] (See Appendix A for filter descriptions)
Stem + Inception-V1	2x{CNN,Batch Norm, Hard Tanh}, 5x{CNN, Batch Norm, ReLU}, 2x{MaxPool, CNN, Batch Norm, ReLU}, 4x{CNN, Batch Norm, ReLU}, {MaxPool, CNN, Batch Norm, ReLU}, 5x{CNN, Batch Norm, ReLU}, 2x{MaxPool, CNN, Batch Norm, ReLU}, 4x{CNN, Batch Norm, ReLU}, {MaxPool, CNN, Batch Norm, ReLU}, 5x{CNN, Batch Norm, ReLU}, {MaxPool, CNN, Batch Norm, ReLU}, 5x{CNN, Batch Norm, ReLU}, {MaxPool, CNN, Batch Norm, ReLU}, 5x{CNN, Batch Norm, ReLU}, {MaxPool, CNN, Batch Norm, ReLU}, 5x{CNN, Batch Norm, ReLU}, {MaxPool, CNN, Batch Norm, ReLU}, 4x{CNN, Batch Norm, ReLU}, {MaxPool, CNN, Batch Norm, ReLU}, 5x{CNN, Batch Norm, ReLU}, {MaxPool, CNN, Batch Norm, ReLU}, Average Pool, Dropout, Flatten, Linear	
Stem + EfficientNet	2x{CNN,Batch Norm, Hard Tanh}, 4x{CNN, Batch Norm, ReLU}, 2x{CNN, Batch Norm, SiLU}, {Average Pool, CNN, SiLU}, {CNN, Sigmoid, CNN, Batch Norm}, {CNN, Batch Norm, SiLU}, {Average Pool, CNN, SiLU}, {CNN, Sigmoid, CNN, Batch Norm}, 21{2x{CNN, Batch Norm, SiLU}, {Average Pool, CNN, SiLU}, {CNN, Sigmoid, CNN, Batch Norm}}, {CNN, Batch Norm, SiLU}, Average Pool, Dropout, Flatten, Linear	

4

[4]Section written by Shamika Dhuri (sdhuri).

4 Dataset

To evaluate our proposed improvements and compare their performance with the original method, we utilized the AI Recognition Dataset available on Kaggle. This dataset contains both AI-generated and real images. The AI-generated images were produced using DALL-E and Midjourney, whereas the real images are confirmed to be created by humans. Most of the AI images are artistic in nature and not photorealistic. This selection is based on findings from previous studies indicating that datasets with a higher proportion of artistic works compared to photographs in the real image category tend to yield better testing results[1].

The original dataset consists of 17,900 fake images and 3,780 real images. To ensure balance, the number of fake images was reduced to equal the number of real images. For training the model, each instance comprised either a fake or a real image that was resized to 256x256 pixels, converted to grayscale, and labeled as either 'real' or 'fake'. [5]

5 Evaluation metric

As the model's output is a binary classification and false positives have the same weight as true negatives, using accuracy is sufficient for model evaluation. Accuracy can explain exactly what percentage of the images were classified correctly as AI generated or real.
The accuracy calculation can be made with the following equation. Given a fingerprint function $T(\cdot)$ and a classification function $F(\cdot)$, the loss can be computed over N images as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

In this equation, TP are true positives, or AI generated images that are classified as AI generated images, TN are true negatives, or real images that are classified as real images, FP are false positives, or real that are classified as AI generated images, and FN are false negatives, or AI generated images that are classified as real images. [6]

6 Loss function

Given the binary output of the model, a simply cross entropy loss is suitable for the training procedure. Given a fingerprint function $T(\cdot)$ and a classification function $F(\cdot)$, the loss can be computed over N images as:

$$L = \sum_{i=1}^{N} y_i log(F(T(x_i))) - (1 - y_i) log(1 - F(T(x_i)))$$

The model performance can then be evaluated as simple error rate where a probability greater than 50% is confirmed as real and less than 50% is predicted as fake. This function can be called $P(\cdot)$ Thus, a correct example has the condition $y = P(F(T(x_i)))$. This error can be computed as:

$$Error = \frac{1}{N} \sum_{i=1}^{N} |y_i - P(F(T(x_i)))|$$

[7]

7 Baseline Selection

In the evolving landscape of AI-generated image detection, it is crucial to employ a baseline methodology that not only reflects the current state-of-the-art but also addresses the inherent challenges posed by the sophistication of generative models. Our selection of "PatchCraft: Exploring Texture Patch for Efficient AI-generated Image Detection" [12] the baseline for our study stems from its innovative approaches and its demonstrated efficacy over previous methods.

[5] Code written by Shamika Dhuri (sdhuri). Dataset description written by Juan Hernandez Gomez (juanh)
[6] Evaluation metric section written by Shamika Dhuri (sdhuri).
[7] Loss function section written by Bain McHale (dmchale).

Firstly, "PatchCraft" introduces the novel Smash/Reconstruction technique, which disrupts the semantic continuity of images to accentuate texture regions. This method is not just a departure from traditional detection mechanisms that rely heavily on semantic information but is a strategic refocusing on the inter-pixel correlation—a fundamental attribute that generative models struggle to replicate accurately. By altering the detector's focus to texture, PatchCraft effectively exploits a consistent and universal weakness across various AI-generated images, thereby enhancing detection generalization.

Secondly, the concept of a universal fingerprint proposed by PatchCraft marks a significant advancement in the field. Unlike previous models that might utilize diverse and potentially model-specific artifacts for detection, PatchCraft leverages the intrinsic contrast in inter-pixel correlations between texture-rich and texture-poor regions within images. This approach is underpinned by the observation that while generative models can create visually convincing fake images, they falter in maintaining the natural textural diversity present in authentic images—a discrepancy that PatchCraft capitalizes on.

Moreover, PatchCraft's comprehensive benchmark for AI-generated image detection, including an array of generative models, sets a new standard for evaluating detection techniques. This benchmark allows for a clear, fair, and replicable comparison between different approaches, ensuring that advancements in the field are measurable and meaningful.

The superiority of PatchCraft over existing baselines is not merely theoretical but has been empirically validated through extensive testing. Its ability to outperform state-of-the-art detectors by a clear margin underlines its effectiveness and the relevance of its foundational concepts. The emphasis on the universal fingerprint of fake images, derived from the inherent weakness in distribution approximation by generative models, ensures that PatchCraft remains robust even against novel or unseen fake image generators.

Therefore, the selection of PatchCraft as our baseline is grounded in its methodological innovation, its focus on universal and inherent flaws of generative models, and its proven superiority over existing techniques. This approach not only aligns with our project's objectives but also sets a formidable benchmark against which the effectiveness of our proposed methods can be rigorously evaluated. On the dataset [1] we established the baseline accuracy by replicating the architecture of the baseline [12]. The extensions that we implemented are explained in detail in the following section. [8]

8 Baseline Implementation and Completeness

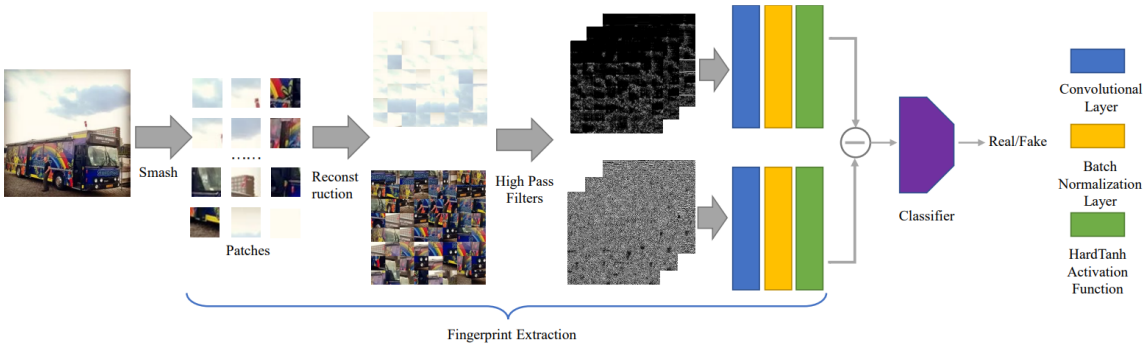


Figure 3: End-to-End workflow of PatchCraft

Our Github Repository has the complete implementation of the baseline, which can be executed by setting the parameters to those from the original Patchcraft paper. They used the label of the image as the prompt to generate fake images for some text2image generative models like DALL-E 2 and Midjourney and we used a Kaggle dataset[8] which was curated from DALLE and Midjourney

[8] Baseline selection done by Sai Deepa Vaddi (svaddi) and Shamika Dhuri (sdhuri). Baseline selection section written by Bain McHale (dmchale).

and has mentioned details in dataset section were used for training the model. For a learning rate of 2e-3 for 30 epochs using the default parameters from the paper insofar as they were included. This resulted in an initial validation accuracy of 85% on our dataset. The paper has an accuracy of 90.12% for Midjourney and 96.60% for DALLE2. We believe the source of difference is our dataset, which differed from the PatchCraft authors. We were unable to find their dataset. First, the dataset we use has a "real" images section of not only photographs, but also digital and painted art. As a result, some of "real" images are significantly more difficult to classify than we presume the images in their dataset are. During the training process, we saw that the identification on the AI generated images was greater than 90%. We saw that the performance on the real image portion of the dataset was <80%. We did not explicitly run tests on each section, but we did visually see these results during training. Additionally, we believe that hyper parameter optimization could affect the results. The original paper did not include optimizers, initial learning rates, schedulers, etc. We did not run ablations on this hyper parameters to find more optimal values.

The following sections mention all aspects of the model pipeline and the implementation to the best of our knowledge. The pipeline starts with images segmented into patches and split into high-texture and low-texture patches. These patches are concatenated, passed through high pass filters, and then subtracted and given to a classifier which in this case is ResNet50. [9]

8.1 Fingerprint Generation

In order to promote generalizability of the model, the first stage of the model generates a fingerprint of every image prior to feeding it into a classifier portion of the model. The idea is that fingerprints generated across different models will retain the same identifiable artifacts of AI generations, but the other model-specific irregularities will be purged. Durall et al. [3] showed that image generation models have certain inter-pixel correlations as a result of their up-sampling step, so the objective is to design a fingerprinting that retains this information. This novel fingerprinting introduces the idea of smash & reconstruction as well filtering approaches. These combine two create a fingerprint that emphasize inter-pixel correlations that can be exploited by the classifier. [10]

8.1.1 Smash & Reconstruction

The smash step serves to remove semantic information from the model. It will randomly crop MxM subsections of the image into patches. By smashing the image this way, it should prevent the model from learning on semantic information like specific parts of facial feature. This way, the model should generalize to new semantic topics that it wasn't trained on. This is accomplished from an LxH image by selecting a random location where $i \in [0 : L - M]$ and $j \in [0 : H - M]$ in the image and cropping out the patch $X_{i:i+M,j:j+M}$ on every channel.

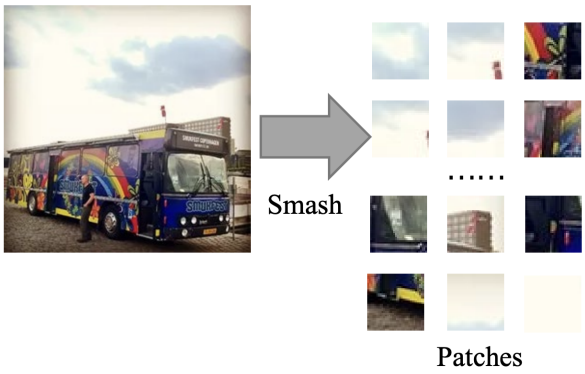


Figure 4: Smashing an image of a bus into patches.

Once the image is smashed, it will be reconstructed into two new images. One image is a combination of patches for the rich texture regions and another is a combination of patches for the poor texture

[9] Code written by Shamika Dhuri (sdhuri). Section written by Sai Deepa Vaddi (svaddi) and Bain McHale (dmchale).

[10] Section written by Bain McHale (dmchale).

regions. A rich texture region is a region where the inter-pixel correlation is low. In other words, the pixels are very different from nearby pixels. This is important because AI generated images tend to have a higher variation on texture richness than real images. The richness r is evaluated by summing the inter-pixel correlations for four directions: horizontal, vertical, diagonal, and cross-diagonal.

$$[1 \quad -1] \qquad \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$r_h = \sum_{i=0}^{M-1} \sum_{j=0}^{M} |x_{i,j} - x_{i+1,j}| \qquad r_v = \sum_{i=0}^{M-1} \sum_{j=0}^{M} |x_{i,j} - x_{i+1,j}|$$

Horizontal Vertical

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \qquad \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

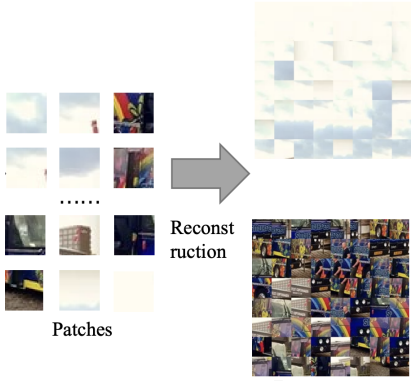
$$r_d = \sum_{i=0}^{M-1} \sum_{j=0}^{M} |x_{i,j} - x_{i+1,j}| \qquad r_{cd} = \sum_{i=0}^{M-1} \sum_{j=0}^{M} |x_{i,j} - x_{i+1,j}|$$

Diagonal Cross-Diagonal

$$r = r_h + r_v + r_d + r_{cd}$$

Figure 5: Calculation of patch texture richness and corresponding filter visualizations.

Once the richness of each patch is calculated, they can be sorted by richness to learn the rich and poor texture regions. From these texture regions, you can reconstruct new images from high texture patches and low texture patches. For each patch P_i in the sorted list P you can fill in a high texture image from top left to bottom right and a low texture image from top left to bottom right. To create the patchworks from DxD patches, you will use the top D^2 images from P and bottom D^2 images from P. [11]



Low Richness Patchwork Image $P_r = \begin{bmatrix} P_{-1} & \dots & P_{-D} \\ \vdots & \ddots & \vdots \\ P_{-D(D-1)-1} & \dots & P_{-D^2} \end{bmatrix}$

High Richness Patchwork Image $P_h = \begin{bmatrix} P_0 & \dots & P_{D-1} \\ \vdots & \ddots & \vdots \\ P_{D(D-1)} & \dots & P_{D^2-1} \end{bmatrix}$

Figure 6: Reconstruction of image from patches sorted in descending richness.

8.1.2 High Pass Filters

Then, a variety of high-pass filters are applied to these patchwork images. The high pass filters are adopted from Fridrich et al. [4] and are designed to extract the noise from the image, which further

[11] Code written by Juan Hernandez Gomez (juanh). Section written by Bain McHale (dmchale).

suppresses all of the semantic information, forcing the fingerprint to only contain artifacts related to how the model was generated instead of what was generated. These high-pass filters should also highlight the inter-pixel relationships for the images. [12]

8.1.3 Fingerprinting Convolutional Layer

The final step in texture extraction is to use some convolutional layers on these filtered images. The motivation here is that there may be additional filtering that could contribute to creating a good fingerprint, and so allowing the model to learn these kernels instead of using preset kernels (like we did with the high pass filters) would be more optimal. This convolutional layer is followed by a batch normalization layer and a hard tanh function. [13]

8.1.4 Texture Region Subtraction

At this point, there are two data points, one for the rich texture image and one for the poor texture image. Before passing the data into the classifier, they are subtracted point-wise to create a singular input. Thus, denoting the Fingerprinting convolutional layers as $C(\cdot)$, the fingerprinting function $T(\cdot)$ is computed as:

$$T(x) = C(P_r(x)) - C(P_h(x))$$

[14]

Figure 7: Ablations.

9 Experiments

After thoroughly reading the paper for the baseline model, we selected four main points to focus on as extensions.

9.1 Modifications on the final classifier

In the paper, the authors mention that the final classifier shouldn't significantly impact the solution's quality. Considering that interpixel correlation is a noticeable fingerprint, a simple classifier should be able to differentiate AI-generated images from real ones.

Nevertheless, pooling layers in a neural network can affect the classifier's access to interpixel correlation information. Altering the network architecture may, therefore, substantially influence the solution's final performance.

Based on this, we experimented with alternative architectures by replacing the ResNet-50 classifier with EfficientNet and Inception models. [15]

[12] Code written by Sai Deepa Vaddi (svaddi). Section written by Bain McHale (dmchale).

[13] Code written by Shamika Dhuri (sdhuri). Section written by Bain McHale (dmchale).

[14] Code written by Shamika Dhuri (sdhuri). Section written by Bain McHale (dmchale).

[15] Code written by Bain McHale (dmchale), Juan Hernandez Gomez (juanh), and Shamika Dhuri (sdhuri). Section written by Bain McHale (dmchale).

9.2 Changes in filters

In the baseline model, the choice of filters is crucial for performance. The model employs specific high-pass filters from SRM to reduce the semantic content, details that could be mimicked in fake images, and to highlight patterns of pixel relationships. This focus on pixel relationships rather than image semantics is key for the effectiveness of the final classification, as it provides the deeper layers of the network with more relevant information for distinguishing between genuine and manipulated images.

Considering this, we experimented with subsampling filters of the type the model currently uses, as well as introducing entirely new filters. [16]

9.3 Medium diversity patches

The current solution classifies patches based on their texture diversity into rich and poor texture parts. Then, a reconstruction step builds new images with low semantic information using patches of a single kind. The final classifier compares the images of each type and assigns a label based on their differences.

The paper mentions that this approach does not work well for models that take real images and make few changes, such as CycleGAN and GauGAN. Therefore, we added a third medium diversity class to gain additional information about the interpixel correlation fingerprint. This allowed us to generate a third type of image, which we compared with the rich and poor diversity images to provide further insights to the final classifier. [17]

9.4 Changes in the subtraction step

The Smash&Reconstruction approach includes a subtraction step just before the final classification. This step takes the output generated by the convolutional blocks for the rich and poor texture images and estimates the difference to label the image.

The subtraction could be losing valuable information that could be used to improve the performance of the final classifier. Therefore, we removed this step and checked how the classifier performed using the output of the previous layers. [18]

[19]

[16] Code written by Shamika Dhuri (sdhuri), Bain McHale (dmchale) and Sai Deepa Vaddi (svaddi). Section written by Bain McHale (dmchale).

[17] Code written by Bain McHale (dmchale). Section written by Bain McHale (dmchale).

[18] Code written by Bain McHale (dmchale). Section written by Bain McHale (dmchale).

[19] Experiments description written by Juan Hernandez Gomez (juanh)

10 Results [20]

Figure 8: Loss and accuracy for training and validation across different classifiers

Figure 9: Loss and accuracy for training and validation across different filter groups

[20] Section written by Bain McHale (dmchale).

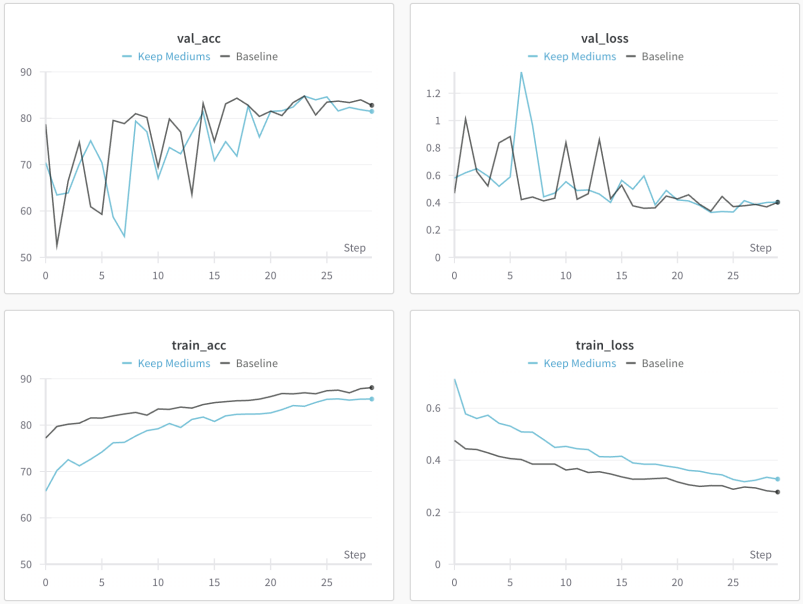


Figure 10: Loss and accuracy for training and validation for including the medium textures

Figure 11: Loss and accuracy for training and validation when skipping the subtraction step

Figure 12: Loss and accuracy for training and validation for final combination of all hyperparameters

11 Discussion [21]

11.1 Modifications on the final classifier

When changing the classifier from Resnet50 to Inceptionv1 and EfficientNet, all performances were withing 1% of each other. The alterations did lead to slight improvements, with inception performing the best. This is unsurprising, as Inception is a more modern architecture that implements the same ideas of Resnet's residual connections, but it additionally introduces its own optimizations. The small difference in performance potentially means that the difficulty of classifying fingerprints does not necessitate such architectural improvements when compared to tasks like raw image classification.

11.2 Changes in filters

When applying the High Pass Filters, the filters results are averaged together prior to being passed into the stem and classifier. As a result, applying too many filters and averaging all the results seems to potentially lose information during the averaging. This was seen in the results of our experiment. Simply adding the Gabor filter as an additional filter did not help. However, taking subsets of the initial filtering section was able to improve performance. This implies that the averaging of the filter results must cancel out some useful information.

11.3 Medium diversity patches

Following the logic that providing more information usually isn't worse, we decided to try including the medium diversity patches into the classifier instead of just the high and low diversity patches. This performance was noticeably worse than the baseline, both in validation and training. This result is surprising given that all the information available in the baseline is also available in this variation. Our explanation is that the model tries to fit to this additional data, but the data is effectively just noise that the model wastes parameters trying to fit to. We believe that, with enough training and tuning, this model could eventually reach the same performance as the baseline once it learned to disregard the medium patches.

[21] Section written by Bain McHale (dmchale).

11.4 Changes in the subtraction step

Removing the subtraction step created a notable improvement in performance. This can be attributed to keeping more information in the fingerprint. The model can learn this simple subtraction step internally within the classifier if it is an important transformation, so there was no need to manually implement it beforehand and lose information.

11.5 Combination of all extensions

We combined the best extension hyperparameters into one final model, which we trained against the baseline. This model did outperform the baseline, but not by a margin larger than some of the extensions individually. This said, it did have a smaller gap between the training accuracy and validation accuracy than other models that reached higher performance. Given that it had not yet overfit at 30 epochs, using different schedulers and training it longer could likely allow it to keep this close validation/train difference when it reaches a higher training accuracy. This discrepancy in final performance compared to the individual extensions can also be attributed randomness in initialization.

12 Future work [22]

12.1 Voting Random Re-crops

We believe there is significant variation in performance as a result of the random crops. If a portion of an image is over or under represented in the reconstructed images, then it is never used in the classifier. We think that, at inference time, an image could be loaded in 3 times with different random crops, then a majority vote could be taken of the outputs. This should reduce the likelihood of poor cropping randomness at inference time.

12.2 Filter Stacking

We believe many of the complications with filters was a result of the averaging mechanism for combining them. Perhaps stacking the filters together and passing them in as their own channels would stop this behavior. While it Comes at a computational cost and may create redundancy, the results may improve.

12.3 Training Length

Our training process involved training each potential ablation for 30 epochs. While this did lead to over fitting in many circumstances, some ablations were still showing an upward trajectory by the end of the 30th epoch. Simply extending the training length could create superior results.

13 Conclusions

In this paper, we introduce enhancements to the AI-generated image detection framework originally proposed in "PatchCraft: Exploring Texture Patch for Efficient AI-generated Image Detection". Our experiments focus on key components of the initial method, including the final classifier, high pass filters, and the subtraction step. We found that modifying the original filters through subsampling, eliminating the subtraction step, and using the Inception model as the final classifier improves performance in detecting fake images. Additionally, testing both the original method and its modifications on the Kaggle AI Recognition Dataset confirms the efficacy of fingerprint-based approaches. The improvements over the baseline highlight significant opportunities to further increase accuracy in fake image detection. [23]

[22] Section written by Bain McHale (dmchale).

[23] Conclusions written by Juan Hernandez Gomez (juanh)

References

[1] Ai recognition dataset. *Kaggle*, 2024.

[2] Xiuli Bi, Bo Liu, Fan Yang, Bin Xiao, Weisheng Li, Gao Huang, and Pamela C. Cosman. Detecting generated images by real images only, 2023.

[3] Ricard Durall, Margret Keuper, and Janis Keuper. Watch your up-convolution: Cnn based generative deep neural networks are failing to reproduce spectral distributions, 2020.

[4] Jessica Fridrich and Jan Kodovský. Rich models for steganalysis of digital images. *IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY*, 7:868, 2012.

[5] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014.

[6] Weize Quan, Kai Wang, Dong-Ming Yan, and Xiaopeng Zhang. Distinguishing between natural and computer-generated images using convolutional neural networks. *IEEE Transactions on Information Forensics and Security*, PP:1–1, 05 2018.

[7] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2022.

[8] SUPERPOTATO9. Ai recognition dataset. *Kaggle*, 2024.

[9] Zhendong Wang, Jianmin Bao, Wengang Zhou, Weilun Wang, Hezhen Hu, Hong Chen, and Houqiang Li. Dire for diffusion-generated image detection, 2023.

[10] Ziyi Xi, Wenmin Huang, Kangkang Wei, Weiqi Luo, and Peijia Zheng. Ai-generated image detection using a cross-attention enhanced dual-stream network, 2023.

[11] Ye Yao, Zhuxi Zhang, Xuan Ni, Zhangyi Shen, Linqiang Chen, and Dawen Xu. Cgnet: Detecting computer-generated images based on transfer learning with attention module. *Signal Processing: Image Communication*, 105:116692, 2022.

[12] Nan Zhong, Yiran Xu, Sheng Li, Zhenxing Qian, and Xinpeng Zhang. Patchcraft: Exploring texture patch for efficient ai-generated image detection, 2024.

[13] Nan Zhong, Yiran Xu, Zhenxing Qian, and Xinpeng Zhang. Rich and poor texture contrast: A simple yet effective approach for ai-generated image detection, 2023.

[14] Mingjian Zhu, Hanting Chen, Mouxiao Huang, Wei Li, Hailin Hu, Jie Hu, and Yunhe Wang. Gendet: Towards good generalizations for ai-generated image detection, 2023.

14 Appendix

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(a)

$$\begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & -3 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(b)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -2 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(c)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 2 & -4 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(d)

$$\begin{bmatrix} -1 & 2 & -2 & 2 & -1 \\ 2 & -6 & 8 & -6 & 2 \\ -2 & 8 & -12 & 8 & -2 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(e)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 2 & -4 & 2 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(f)

$$\begin{bmatrix} -1 & 2 & -2 & 2 & -1 \\ 2 & -6 & 8 & -6 & 2 \\ -2 & 8 & -12 & 8 & -2 \\ 2 & -6 & 8 & -6 & 2 \\ -1 & 2 & -2 & 2 & -1 \end{bmatrix}$$

(g)

Figure 13: Filters a, b, c, d, e, f, and g

Filter h: OpenCV's Gabor Filters with parameters: Kernel Size of 20, Sigma of 4, Lambda of 10, Gamma of 0.5, Psi 2, and Theta of 2 [24]

[24] https://shimat.github.io/opencvsharp_docs/html/ee0bad70-d36c-2c01-7be7-631cb6f01746.htm